

ADaM Series: 05- Datasets Aren't the Deliverable: Analysis Results Metadata and the ARS Connection

Varun Debbeti · Clinical Programmer

ADaM IN PRACTICE · A TECHNICAL SERIES FOR CLINICAL TRIAL STATISTICAL PROGRAMMERS

Where ADaM ends and the CDISC Analysis Results Standard begins, and why building results metadata upstream changes how you structure ADaM in the first place.

Abstract

Statistical programming has standardized every stage of the clinical data pipeline except the one a reviewer cares about most: the analysis results themselves. A number in a submitted table has traditionally been defined only by an RTF page and the SAS program behind it. This article examines two CDISC standards that close that gap. Analysis Results Metadata (ARM) v1.0 attaches result-level metadata to define.xml and is established submission practice today. The Analysis Results Standard (ARS) v1.0, published in April 2024, goes further: it defines each analysis as a machine-readable object whose selectors name real ADaM datasets, variables, and values. Because ARS can only express what exists as data, it exposes every derivation hiding in TFL programs as unfinished ADaM design. The argument, illustrated with a worked safety-table example, is that building results metadata upstream changes how you design datasets, and that the deliverable was never the dataset but the defensible, reproducible result.

1. Introduction

For twenty years, CDISC standards have marched steadily down the clinical data pipeline: CDASH for collection, SDTM for tabulation, ADaM for analysis-ready data. The march stops just short of the finish line. The results in a submitted table, the n, the percentage, the p-value, have no standard representation of their own; they exist as ink on a page plus the program that produced them. When a reviewer asks exactly which population, which records, and which method produced a given cell, the answer must be reconstructed from shells, programs, and memory. For statistical programmers this is not an abstract gap. It is the source of

double-programming discrepancies, shell-interpretation meetings, and analysis logic that quietly escapes the dataset into table programs where define.xml cannot see it.

This article traces how two CDISC standards close that gap and what they demand of ADaM design. Section 2 describes the gap after the last ADaM dataset and introduces ARM and ARS. Section 3 walks through what ARS v1.0 actually models. Section 4 takes the analysis-ready contract literally and shows how ARS audits your ADaM. Section 5 works through a single safety table specified end to end. Section 6 covers ARM v1.0 as the bridge to build today, and Section 7 lays out a pragmatic adoption sequence for this year. Section 8 concludes.

2. The gap after the last ADaM dataset

Our discipline has spent twenty years standardizing everything up to the analysis: collection (CDASH), tabulation (SDTM), and analysis-ready data (ADaM). Then the pipeline hits the results themselves (the numbers in Table 14.3.1.1) and standardization stops. The result of an analysis has traditionally existed only as ink on an RTF page plus the SAS program that produced it. When a reviewer asks *what population, what parameter, what method produced this cell*, the answer lives in a programmer’s head, a TFL shell, and a program that may or may not still run. Analysis results were, in effect, the last unstandardized objects in the submission.

Two CDISC artifacts address this. The older one, **Analysis Results Metadata (ARM) v1.0 for Define-XML v2**, attaches result-level metadata (analysis reason, dataset, selection criteria, documentation links) to define.xml for key efficacy and safety analyses: useful, submitted with real NDAs, but descriptive after the fact. The newer and far more consequential one is the **CDISC Analysis Results Standard (ARS) v1.0**, published in April 2024: a logical model that describes an analysis completely and machine-readably *before* any code exists. ARS is not a dataset standard; it is a metadata standard for the analyses themselves, and it moves the boundary of what statistical programmers should consider 'the deliverable.'

3. What ARS v1.0 actually models

The ARS model organizes a reporting effort into a ReportingEvent (e.g., the CSR TFL package) containing Analyses and Outputs. The pieces a programmer touches daily map onto it cleanly:

ARS object	What it captures	Today’s informal equivalent
AnalysisSet	Population selection as executable criteria (e.g., ADSL.SAFFL="Y")	Shell footnote + programmer memory
DataSubset	Record-level selection (e.g., TRTEM-FL="Y", ANL01FL="Y")	WHERE clauses scattered across programs

GroupingFactor	Row/column groupings (treatment via TRT01A, severity via ASEV)	Hard-coded BY/CLASS statements
AnalysisMethod / Operation	The statistical operation(s): n, %, mean, CI, hazard ratio, each with a defined result pattern	The PROC step
Analysis	Binding of method + set + subset + groupings against a named ADaM dataset/variable	The program itself
Output / result cells	Each result value, addressable by its group combination	A number on an RTF page

Table 1. ARS v1.0 core objects. The JSON/YAML serializations are published with the standard (cdisc-org/analysis-results-standard on GitHub).

The decisive property: every selector in the model points at **named ADaM content: a dataset**, a variable, a value. An ARS file for a safety summary does not say “treatment-emergent AEs”; it says ADAE where TRTEMFL='Y', analysis set defined by ADSL.SAFFL='Y', grouped by TRT01A against codelist values. Which is exactly why ARS, though formally downstream of ADaM, exerts pressure *upstream* on how you build ADaM.

// Excerpt: ARS JSON for one analysis of Table 14.3.1.1 (AE overview)

```
{
  "id": "An01_05_SAF_Summ_TEAE",
  "name": "Subjects with any TEAE - summary by treatment",
  "analysisSetId": "AnalysisSet_02_SAF",
  "dataset": "ADAE", "variable": "USUBJID",
  "orderedGroupings": [{ "groupingId": "AnlsGrouping_01_Trt" }],
  "dataSubsetId": "Dss01_TEAE", // TRTEMFL EQ 'Y'
  "methodId": "Mth01_CatVar_Count_ByGrp", // n and % operations
  "referencedAnalysisOperations": [
    { "referencedOperationRelationshipId": "denominator",
      "analysisId": "An01_05_SAF_BigN" } // % denominator is itself
  ] // a traceable analysis
```

}

4. Where ADaM ends: taking “analysis-ready” literally

ADaM’s contract has always been that the dataset supports the analysis with minimal manipulation, the “one PROC away” ideal. ARS operationalizes that contract, because its selectors can only express what exists as data: flags, variables, parameter codes. Every place your TFL programs do work that ARS cannot describe is a place your ADaM stopped short of analysis-ready. The audit is brutally simple: read your TFL programs and list every data step between the ADaM input and the PROC. Each one is either (a) a derivation that belongs in ADaM as a flag, parameter, or category variable, or (b) presentation logic (ordering, labels, decimal alignment), which is legitimately the output layer’s job. In our experience auditing legacy TFL suites, the split runs roughly 70/30 in favor of (a): population definitions recomputed in table programs, ad-hoc severity regroupings, on-the-fly baseline recalculations. All of it is analysis logic that escaped the dataset and its metadata, invisible to define.xml and unreproducible from the submitted package.

Concretely, designing ADaM with ARS in view changes several habits. Analysis populations exist *only* as ADSL flags, never as WHERE clauses invented per table. Record selection for each display is an ANLzzFL or criterion flag with define.xml documentation, so a DataSubset can reference it by name. Grouping schemes (dose groups, severity collapses, age bands) become permanent category variables with controlled values rather than PROC FORMAT conveniences. And numerators/denominators that differ across tables stop being programmer folklore: each is a named analysis with a referenced-operation relationship, as in the JSON above.

5. A worked example: one table, fully specified

A Phase III trial in atopic dermatitis (~800 subjects, two doses plus placebo) piloted ARS on its safety summary package. For the AE overview table, the exercise of writing the ARS file *before programming* surfaced exactly the ambiguities that normally surface during double-programming discrepancy resolution: whether the denominator for the ~~Q~~-TEAE percentage was the safety population or subjects with any exposure record (they differ by two subjects); whether “related” meant investigator-assessed only or the sponsor-upgraded relationship variable; and which MedDRA version governed a mid-study uplift. Each ambiguity became a named object (an AnalysisSet, a DataSubset against a specific ADaM variable, a documented WhereClause) rather than a footnote negotiation. The team’s assessment afterward: the ARS file took roughly two days to author with a template library, and eliminated an estimated week of downstream discrepancy resolution and shell-interpretation meetings. That trade only improves as template libraries mature: the CDISC eTFL portal and the open-source TFL Designer tooling ship ARS-native templates for common safety displays.

6. ARM v1.0 today: the bridge you should already be building

While ARS matures, Analysis Results Metadata for Define-XML v2 is the established, submit-today mechanism, and it is the right on-ramp, because it forces the same discipline at lower cost. ARM attaches a

<arm:AnalysisResultDisplays> section to define.xml in which each key display carries: the analysis reason (e.g., PRIMARY EFFICACY, per controlled terminology), the analysis purpose, the datasets and variables consumed, the selection criteria as machine-readable where-clauses, and links to the SAP section and the generating program. Sponsors have included ARM for primary and key secondary analyses in NDA/BLA packages for years, and reviewers use it as the index into the efficacy datasets.

The practical guidance mirrors VLM discipline from the parameter-management world: cover the analyses a reviewer will interrogate first (primary, key secondaries, and any analysis feeding a labeling claim) rather than attempting all several hundred displays; make the where-clauses real (referencing actual flag variables and values, not prose); and generate the ARM section from the same metadata source that drives your TFL specs, so it cannot drift from what the programs actually did. Teams that author ARM by hand at the end of a study reliably produce criteria that disagree with the table programs, which is worse than no ARM at all, because it documents the disagreement.

The relationship between the two standards is generational, not competitive: ARM describes results *after the fact* inside define.xml; ARS defines analyses *before the fact* as first-class objects from which programs, and eventually the ARM section itself, can be generated. If your organization builds the ARM authoring muscle now (named analysis sets, real where-clauses, metadata-sourced generation), the ARS transition becomes a serialization change rather than a process revolution.

A fair objection at this point: is this not just more metadata to maintain, drifting out of sync the way specs always have? The answer depends entirely on generation direction, the same principle that governs parameter registries. ARS files authored after programming, as documentation, will drift and rot. ARS files authored first, from which table programs are generated or against which they are mechanically checked, cannot drift: they are the specification, and the open-source ecosystem forming around the standard (schema validators, template libraries, R and SAS reference implementations in the CDISC pilot repositories) exists to make that authoring cheap enough to do first.

7. What to do about it this year

ARS adoption does not require waiting for your organization to buy anything. A pragmatic sequence:

- **Run the analysis-ready audit** on one recent study's TFL programs. The list of pre-PROC data steps is your ADaM design debt, and fixing it pays off regardless of ARS.
- **Author ARM v1.0 for key analyses now.** Analysis results metadata in define.xml is established practice for primary/key secondary efficacy analyses, and it forces the same population/subset/method discipline in miniature.
- **Pilot ARS on one output family.** **Safety** summaries are ideal because they are templatable. Use the published JSON schema and examples from the CDISC GitHub; validate that every selector in your file names a real ADaM variable and value.
-

Feed the lessons back into ADaM specs: every ambiguity ARS forces you to resolve becomes a flag or variable in the next study's ADaM, specified before programming rather than discovered after.

The direction of travel is unambiguous: CDISC's 360i program is building the connected, metadata-driven pipeline in which ARS is the results layer, and regulatory interest in reproducible, machine-readable analysis definitions is growing on the same timeline. The teams that will find that transition easy are the ones whose ADaM datasets already honor the original contract: everything analytic in the data and its metadata, nothing analytic hiding in the table programs. Datasets were never the deliverable. Defensible, reproducible *results* are, and for the first time, there is a standard for saying precisely what a result is.

8. Conclusion

The through-line of this article is a single shift in perspective: the analysis result, not the dataset, is the deliverable, and there is now a standard for saying precisely what a result is. ARS v1.0 models each analysis as a set of named, machine-readable objects whose selectors point at real ADaM datasets, variables, and values, which means it doubles as an audit of whether your datasets honor the analysis-ready contract. ARM v1.0 is the lower-cost, submit-today on-ramp that builds the same population, subset, and method discipline inside `define.xml`.

The practical program follows directly. Audit one study's TFL programs and treat every pre-PROC data step as ADaM design debt. Author ARM for primary and key secondary analyses from the same metadata source that drives your TFL specs, so it cannot drift. Pilot ARS on one templatable output family, and feed every ambiguity it forces you to resolve back into the next study's ADaM specification. Teams that build these habits now will experience the coming metadata-driven pipeline as a serialization change rather than a process revolution.

References

CDISC Analysis Results Standard (ARS) v1.0, April 2024. Model, JSON/YAML schemas, and examples: cdisc.org/standards/foundational/analysis-results-standard and github.com/cdisc-org/analysis-results-standard.

CDISC Analysis Results Metadata (ARM) v1.0 for Define-XML v2, CDISC.

CDISC ADaM v2.1 and ADaM Implementation Guide v1.3, CDISC, November 2021.

"Selecting GeARS: Analysis Results Standard Implementation Options," PHUSE US Connect 2025, Paper DS09 ([lexjansen.com](https://www.lexjansen.com/phuse/2025/US/DS09)).

CDISC eTFL Portal and TFL Designer (ARS-native template tooling): cdisc.org/kb/etfl.

CDISC 360i program overview, cdisc.org; FDA Study Data Technical Conformance Guide (current edition), [fda.gov](https://www.fda.gov).

